

Архитектура компьютера и основы ОС Linux

Оперативная память

Реальный и защищенный режим. Страничная адресация памяти. Оценка загрузки памяти в Linux.

Оглавление

[Введение](#)

[Разрядность памяти](#)

[Реальный режим 8086](#)

[Операция сдвига](#)

[Сегментные регистры](#)

[Стек](#)

[Сегментная адресация](#)

[Реальный режим](#)

[Реальный режим процессора 80286](#)

[HMA \(High Memory Area\)](#)

[Защищенный режим](#)

[Защищенный режим 80286](#)

[Виртуальная память процессора 80286](#)

[Защищенный режим 80386](#)

[Виртуальная память](#)

[Страничная адресация памяти](#)

[Объем памяти в 32-битных системах](#)

[Виртуальный режим 8086](#)

[DOS в защищенном режиме, EMM386](#)

[Полноценная реализация защищенного режима в MS DOS, DPMI](#)

[Плоская модель памяти](#)

[Оценка загрузки LINUX](#)

[Утилита free](#)

[Практическое задание](#)

[Дополнительные материалы](#)

[Используемая литература](#)

Введение

Определить, чем физически является оперативная память, довольно сложно. Сверхбыстрая оперативная память — регистры процессора, — фактически реализована внутри CPU. Также в составе процессора имеется многоуровневый кеш, получающий код и данные из памяти еще до начала их выполнения.

Во многом оперативная память — это то, что находится в RAM, но не обязательно.

Виртуальная память (файл подкачки, раздел подкачки) позволяет выгрузить часть относительно редко используемых данных на диск.

Таким образом, оперативная память, используемая в системе, распределена по нескольким разнородным устройствам.

Разрядность памяти

Способность процессора работать с тем или иным объемом памяти определяется его разрядностью. В простейшем случае регистр указателя инструкции и регистры для указателя на данные позволяют использовать адреса, не превосходящие максимальное число, которое может быть записано в регистр. Таким образом, максимальный размер памяти, адресуемый регистром X с битностью Y , вычисляется как 2^Y байт: теоретически четырехбитный регистр может адресовать $2^4 = 16$ байт, восьмибитный — уже $2^8 = 256$ байт, а шестнадцатибитный, соответственно, $2^{16} = 65\,536$ байт.

Объем растет экспоненциально.

32-битный $2^{32} = 4\,294\,967\,296$ байт, что равно порядка 4 Гб. Отсюда и ограничение по памяти 32-битных машин, которого нет у 64-битных.

64-битный $2^{64} = 18\,446\,744\,073\,709\,551\,616 = 16\,777\,216$ Тб. Кстати, такое число зерен находится на последней клетке в задаче о зернах на шахматной доске.

Существуют также 128-битные регистры — к слову, 128-битные адреса используются в GUID в GPT, IPv6-адреса имеют размер в 128 бит, современные процессоры содержат 128-битные регистры SSE для операций над числами с плавающей точкой. Впрочем, это еще не говорит, что сам процессор — 128-битный: битность процессора определяется основными используемыми регистрами, операциями над ними и способом манипуляции с памятью. Теоретически возможны и полноценные 128-битные процессоры, а графические процессоры могут использовать и 512-битные регистры. Но фактический предел битности сейчас — показатель в 64 бита.

Если ситуация с 32-битной и 64-битной адресацией понятна — 32-битные машины могут использовать до 4 Гб, а 64-битные свыше 4 Гб, — то 16-битная машина, получается, может задействовать всего 64 Кбайта. Это даже не 640 Кб, которых, как обещал в свое время Билл Гейтс, хватит всем.

Процессор 4004 адресовал до 80 ячеек по 4 бита, но даже для этого не хватило бы содержимого регистра. В этом случае применялось значение пары регистров.

8080 был восьмибитным, но использовал 16-битную шину данных, что позволяло ему адресовать до 64 Кб памяти (и инструкций, и данных). Для этого также использовалась пара восьмибитных регистров, воспринимаемая как один 16-битный. Тем не менее процессор был 8-битным, так как операции выполнялись над 8-битными регистрами. Многие восьмибитные процессоры той эпохи (8080, ZX80 и т. д.) позволяли работать с 64 Кб памяти.

8086 — первый 16-битный процессор. Тем не менее 64 Кбайт памяти было уже мало. 20-битная шина адреса позволяла адресовать $2^{20} = 1\,048\,576 = 1$ Мбайт памяти. Но для адресации 1 Мб уже не хватало

бы даже 16-битного регистра. Для этого была придумана схема «сегмент:смещение», которая надолго определила стиль написания программ для MS DOS. Такой режим позже получил название реального режима 8086.

Реальный режим 8086

Операция сдвига

Одна из наиболее простых в аппаратной реализации арифметических операций в процессорах — операция сдвига. Сдвиг на единицу влево преобразует двоичный 1 в 10 (2), далее в 100 (4), далее в 1000 (8) и т. д. Фактически в этом случае происходит умножение на соответствующие степени двойки. Операция сдвига вправо аналогична операции деления. Существуют разновидности сдвига (арифметический, циклический), но в целом в них действует общая идея.

Сегментные регистры

В 8086 появились сегментные регистры:

- CS (Code Segment) — сегмент кода;
- DS (Data Segment) — сегмент данных;
- ES (ExtraSegment) — дополнительный сегмент;
- SS (Stack Segment) — сегмент стека.

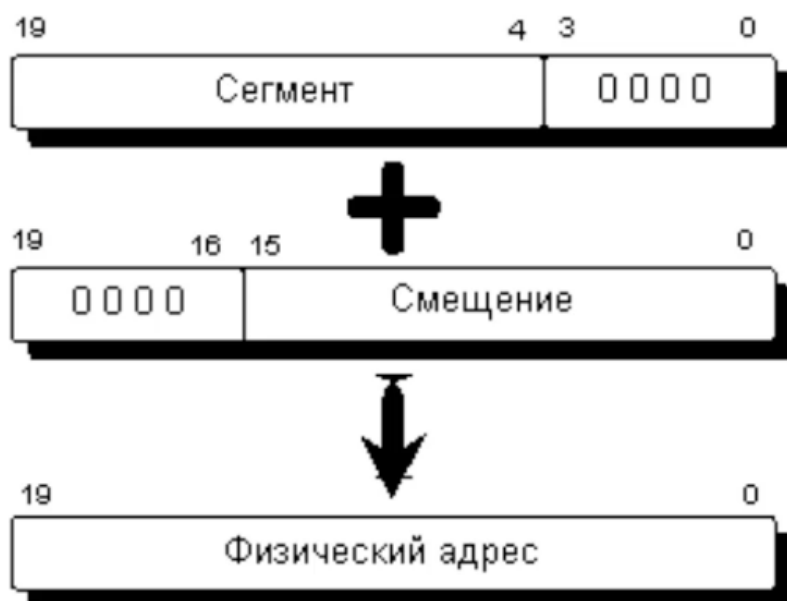
Сегмент CS использовался для адресации к коду программы, DS и ES — для данных, SS — для стека.

Стек

Стек — это особый способ адресации памяти, так называемый LIFO (Last In, First Out, «последним пришел — первым ушел»). Для работы со стеком используются команды ассемблера **push** (положить на вершину стека) и **pop** (снять с вершины стека). Отдельные операции **pushf** и **popf** позволяли поместить или извлечь из стека флаговые регистры. Стек активно принимает участие в сохранении состояний при вызове функций и прерываний. Так как стек растет только в одну сторону, может произойти ошибка переполнения стека (Stack overflow). Такая ситуация может случаться при применении рекурсии и неизбежно произойдет при бесконечной рекурсии, что вызовет переполнение процессора. Существуют языки программирования и процессоры, ориентированные на стек. Это язык Forth и Forth-процессоры.

Сегментная адресация

Что же такое сегменты? Если в 8080-подобных процессорах (ZX80 и т. д.) адресное пространство ограничивалось 64 Кб, то в 8086 для доступа к коду (регистр IP) и данным (например, регистр BX) по-прежнему использовались 16-битные регистры — они позволяли адресовать 64 Кб, но памяти было больше, так как для адресации памяти использовалась 20-битная шина данных. Но так как применялись 16-битные регистры, для организации доступа памяти использовались сегменты, содержащие по 64 Кб. Доступ к ячейке памяти происходил путем адресации к памяти через 20-битную шину данных, поэтому фактический адрес высчитывался благодаря сдвигу влево сегментного регистра на 4 бита (фактически — аппаратное умножение на 16) и прибавление так называемого смещения.



В INTEL-ассемблере сегмент:смещение записывается через двоеточие.

- **mov ax, [bx]** — указание на пересылку значения из памяти в **ds:bx** в регистр **ax**;
- **mov ax, es:[bx]** — явное указание переслать значение из памяти в **es:bx** в регистр **ax**;
- **jmp xxx** — изменение регистра **ip** и переход к коду, находящемуся в памяти по адресу **cs:ip**.

Так, адреса **0x0400:0x0001** и **0x0000:0x4001** будут ссылаться на одну и ту же ячейку памяти, поскольку $0x400 \times 16 + 1 = 0 \times 16 + 0x4001$.

- диапазон адресов: **0000:0000 — FFFF:FFFF**
- рабочий диапазон: **0000:0000 — FFFF:000F**
- адреса начиная с **ffff:0010** ведут к переполнению.

Рассмотрим пример:

Реальный линейный адрес:

+

0000000000000000000010000

=

10000000000000000000 — 21 бит

Более того, попытки таким образом адресовать данные за пределами 1 Мб приводили к обращению к начальной области данных. В 80286 даже понадобилось реализовать режим совместимости с учетом данной «недокументированной возможности» (то есть ошибки).

Достоинства сегментной адресации:

- возможность выделять программам разные сегменты;
- возможность адресовать большее адресное пространство, чем позволяет разрядность процессора.

Недостатки сегментной адресации:

- неоднозначность задания адресом сегмент:смещение адреса, выставляемого на адресную шину для доступа к памяти;
- неизолированность сегментов;
- неинтуитивный механизм формирования.

Реальный режим

Наиболее серьезным недостатком механизма управления памятью в 8086 (а также в 8088 и 80186) была невозможность защиты памяти. Все программы могли влиять друг на друга, не приходилось говорить о пространстве ядра и пространстве пользователя: все приложения имели равный доступ ко всей памяти ЭВМ.

Такой режим не носил определенного названия, и лишь в дальнейшем, с появлением процессора 80286 и «защищенного режима» 286, режим работы 8086 назвали реальным режимом.

Реальный режим продолжал использоваться и в процессорах 80286, 80386, 80486. Именно в нем по умолчанию выполнялась операционная система MS DOS.

Процессор 8086 мог адресовать память размером до 1 Мб. Из них прикладным программам отводилось всего 640 Кб (которых, по Гейтсу, «хватит всем»). Конечно, так продолжаться не могло, и появился процессор 80286.

Реальный режим процессора 80286

Процессор 80286 имел не 20-битную, а 24-битную шину, что позволяло адресовать уже $2^{24} = 16$ Мб памяти. Однако возникли проблемы: например, как быть с совместимостью с привычной багофичей процессора 8086? Был добавлен элемент (Gate 20), который позволял отключить 21 линию адресной шины — таким образом, процессор работал аналогично 8086, обращаясь к начальной памяти. При этом появлялась возможность адресовать до 16 Мб верхней памяти (High Memory Area).

HMA (High Memory Area)

В реальном режиме из 24 бит адресной шины процессора 80286 (и 32 бит процессора 80386) доступны только 20 бит. Тем не менее уже упоминалась ошибка 8086, которая при обращении к сегменту 0xFFFF приводила к обращению к первым байтам памяти. Однако если снять блокировку 21 линии, появлялась возможность адресовать память за пределами 0xFFFF. Дополнительный сегмент (размером 64 Кб — 16 байт) был назван **HMA** (High Memory Area). Драйвер MS DOS **himem.sys** позволял использовать эту

память и при инструкции **DOS=HIGH** помещать ядро MS DOS в HMA. Впервые **himem.sys** появился в Windows 2.1 и служил для разблокировки отключенной по умолчанию линии (A20). В реальном времени появилась возможность использовать HMA.

Защищенный режим

В реальном режиме не было возможности защитить одни программы (память, выполнение прерываний и т. д.) от других, поэтому был разработан защищенный режим. Впервые он появился в 80286 процессоре, но полноценно был реализован уже в 80386

Идея защищенного режима состоит в том, что появилась возможность защищать память, то есть такую работу, что один пользовательский процесс не может повредить данные другого, и никакой из них не может повредить данные ядра.

Кроме того, защищенный режим и в 80286 и в 80386 позволял адресовать все доступное адресное пространство.

Защищенный режим 80286

Процессор 80286 мог работать в двух режимах: реальном, который был полностью аналогичен режиму работы процессора 8086, и защищенном. Если в реальном режиме все ПО, написанное для 8086, могло выполняться без изменений, то в защищенном появился полноценный механизм доступа к 16 Мб памяти.

В защищенном режиме 80286 впервые появились кольца защиты (от 0 до 3).

Физический адрес в 80286 процессоре формировался следующим образом. Внешне адрес так же состоял из двух чисел, но если раньше это было «сегмент:смещение», то теперь «селектор:смещение». В сегментных регистрах хранился селектор, содержащий:

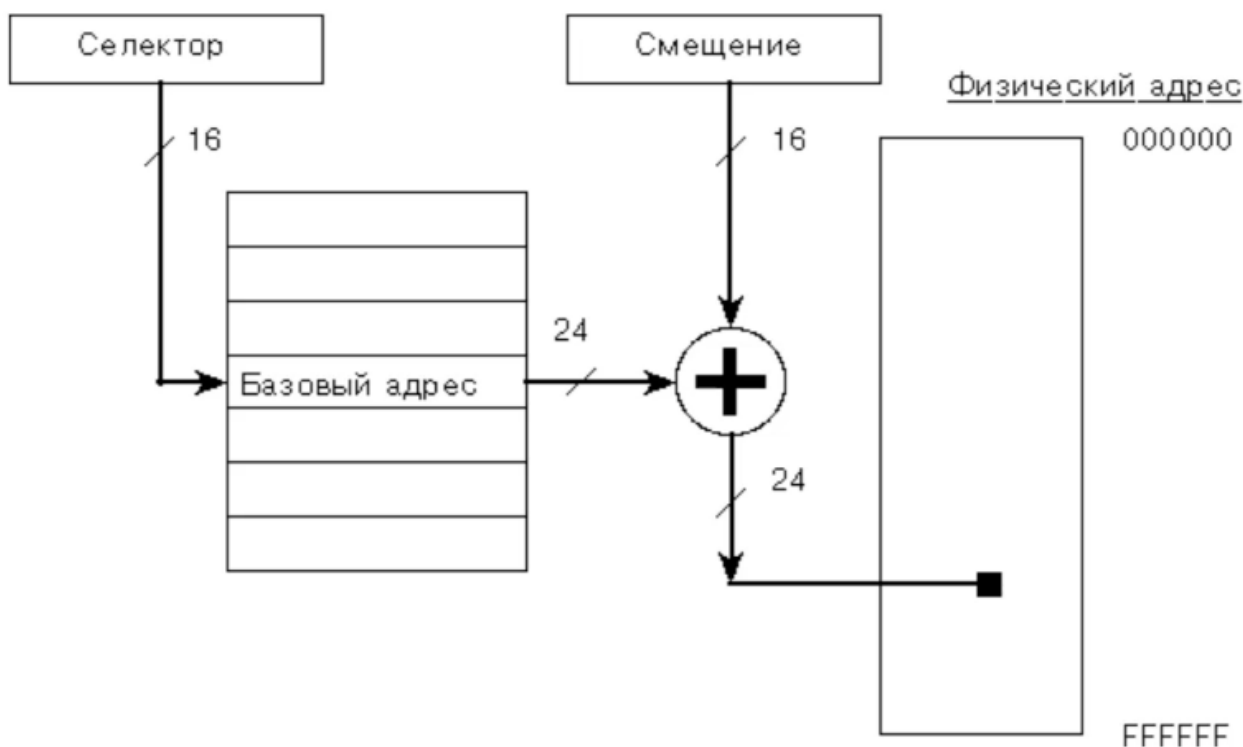
- индекс дескриптора в таблице дескрипторов (13 бит);
- один бит — индикатор таблицы TI, определяющий, к какой таблице дескрипторов будет производиться обращение (к локальной или к глобальной);
- 2 бита (RPL) запрашиваемого уровня привилегий.



Уровень привилегий **RPL** мог содержать 4 значения, от 00 до 11, и определял, в каком кольце защиты будет происходить работа.

Индикатор таблицы **TI** (table indicator) имел значения для указания:

- 0 — **GDT** (Global Descriptor Table) — общий для всех систем. Ее размер и положение определяется регистром GDTR;
- 1 — **LDT** (Local Descriptor Table). Локальных таблиц дескрипторов может быть много, по количеству задач или потоков. Каждая задача может иметь свою LDT. На положение таблицы текущей задачи указывает регистр LDTR.



Далее происходило обращение к соответствующим таблице дескрипторов и дескриптору, который содержал начальный, 24-битный, адрес сегмента, размер сегмента и права доступа. После этого необходимый физический адрес вычислялся путем сложения адреса сегмента со смещением, хранящимся в 16-разрядном указательном регистре.

Фактически защищенный режим 80286 использовал значение сегмента в качестве индекса в таблице дескрипторов. Более того, после перехода в защищенный режим процессора 80286 вернуться в реальный режим можно было только сбросом. В реальном режиме код и прерывания 8086 не могли выполняться. Перед разработчиками возникла дилемма: либо использовать преимущества защищенного режима, но отказаться от совместимости со старым ПО, либо поддерживать совместимость, но работать в реальном режиме.

В защищенном режиме 80286 работали ОС Windows 2.0, OS/2.

Реализация 80286 не позволила Windows выполнять одновременно несколько программ MS DOS и стала одной из причин раздора между Microsoft и IBM, которая настаивала на совместимости OS/2 и процессора 80286.

Виртуальная память процессора 80286

Несмотря на возможность адресовать до 16 Мб, компьютеры редко оснащались более чем 2–4 Мб оперативной памяти. В связи с этим появилась идея подкачки, или свопинга. В то время как программа могла адресовать до 16 Мб, в распоряжении было гораздо меньше памяти. Но какие-то сегменты могли быть выгружены на диск, а при необходимости возвращены в оперативную память, так что появилась возможность использовать для программы больше памяти, чем могла предоставить RAM. Подобным образом механизм виртуальной памяти был реализован в OS/2 (версии 1.0 — 1.3).

Фактически этот механизм на 80286 не был эффективен, и полноценно реализовать его удалось уже на 80386.

Защищенный режим 80386

Полноценный защищенный режим был реализован в Intel 80386. В отличие от 80286, процессор был 32-битным и обладал также 32-битной адресной шиной, что позволяло адресовать до 4 Гб памяти и, более того, отказаться от идеи «сегмент:смещение». 32-битные регистры (как EIP, так и общего назначения) позволяли адресовать память до 4 Гб.

В отличие от 80286, в процессоре 80386 появилась возможность обратного перехода в реальный режим, а также использования аппаратной виртуализации 8086 процессора — «режима виртуального 8086». Это было сделано в интересах компании Microsoft — для поддержки объема программного обеспечения для DOS.

Виртуальная память

В эпоху процессора 80386 обычным объемом RAM было от 2 до 8 Мб. 8Мбит было роскошью.

Виртуальная память — это механизм, который позволял организовать большой объем памяти за счет диска, используя виртуальные адреса. Для трансляции виртуальных адресов в физические применяется особое аппаратное обеспечение — блок управления памятью. Для программы виртуальная память выглядит как доступное непрерывное адресное пространство. Программа не знает, какую она память использует на самом деле. Хранится она в ОЗУ или на диске, в специальном файле или разделе подкачки. Процесс выгрузки данных из оперативной памяти на жесткий диск и загрузки обратно называется свопингом (swapping). Если ваш компьютер «тормозит» при работе сложного ПО, мигает индикатор диска, а программа «подвисает», скорее всего не хватает оперативной памяти.

В этот момент и происходит перемещение данных виртуальной памяти между диском и оперативной памятью. Это возможно благодаря страничной адресации памяти.

Страничная адресация памяти

В Intel 80386 появилась страничная адресация памяти. Теперь память, к которой обращался процессор, могла не соответствовать физической. Это позволило полноценно реализовать как защиту приложений, так и работу с виртуальной памятью.

Единица отображения виртуальных адресов — страница.

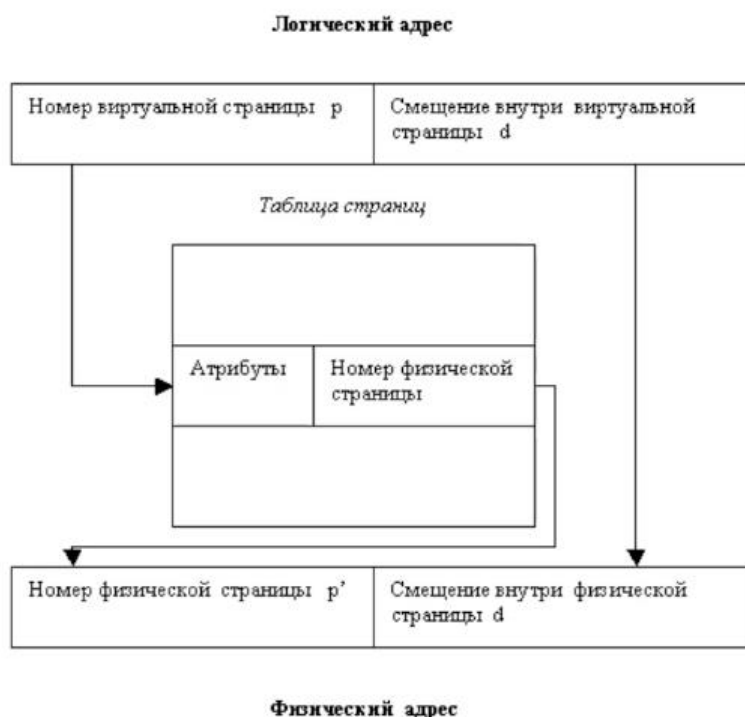
Страница имеет размер 4096 байт.

Страничная адресация позволила решить вопрос фрагментации памяти. Виртуальная память процесса состоит из страниц, часть из которых отображаются на физическую память, а другие могут находиться в swp. При этом в физической памяти страницы не обязаны следовать друг за другом: если процесс освобождает страницу памяти, ее может использовать следующий запросивший процесс.

Также страничная адресация решает проблему изоляции процессов, так как позволяет создать собственное виртуальное адресное пространство для каждого. Также она изолирует области ядра от кода пользовательского режима. Кроме того, страничная адресация поддерживает режим памяти только для чтения и для неисполняемой памяти. Это позволяет реализовать разделенную память и heap (кучу).

Разделенная память хранит библиотеки, необходимые для работы программ.

Heap хранит данные, которые необходимы для процесса, как правило для выделения структур.



Преобразование логического адреса страничной адресации в физический происходит в таблице страниц. Мы указываем номер виртуальной страницы и смещение внутри нее. Номер виртуальной страницы берется из таблицы страниц, который содержит атрибуты страницы и номер физической страницы, к которому и добавляется смещение внутри нее.

Объем памяти в 32-битных системах

Максимальный объем, адресуемый 32-битным регистром, 4 Гб.

Это легко рассчитывается: $2^{32} = 4294967296 = 4 \text{ Гб}$. Но доступная память будет меньше, так как из нее определенный объем должен быть выделен на PCI устройства.

Как же использовать больше, чем 4 Гб оперативной памяти?

Есть два варианта.

Первый и наиболее правильный — использовать 64-битные системы.

Второй — использовать PAE, Physical Address Extension. Эта возможность появилась впервые в Pentium Pro. Позволяла использовать более 4 Гб памяти, но не выделяла более 4 Гб одному процессу. В некоторых ОС от Microsoft данная возможность была принудительно отключена. В Linux поддержка PAE была добавлена в версии ядра 2.3.23 (1999 год).

Виртуальный режим 8086

В 80286 было два режима: защищенный и реальный. В 80386 появился новый режим, который позволил писать приложения в защищенном режиме, но исполнять ПО, написанное для 8086, — режим виртуального 8086. Он стал первой ласточкой технологий аппаратной виртуализации.

Программа, исполняемая в таком режиме, имеет ограничение в объеме памяти, как и в 8086. Тем не менее благодаря страничному отображению эта память может находиться в произвольном месте памяти — и две программы, исполняемые в виртуальном режиме, могут адресовывать разные области памяти, чего не получилось бы в реальном режиме.

Программа выполняется с низким приоритетом, в кольце 3. Благодаря этому прерывания не должны были выполняться, однако фактически это происходило. Вызов прерывания создавал исключение, при котором управление передавалось операционной системе. А она, получив данные о том, какое прерывание пыталась выполнить 8086-программа, могла выполнить это или иное прерывание либо совершить эмуляцию и т. д.

В виртуальном режиме исполняется MS DOS при использовании драйвера EMM386. Именно в виртуальном режиме запускались приложения MS DOS в операционных системах Windows 3.11 и Windows 95 (тот самый «DOS в оконном режиме»).

DOS в защищенном режиме, EMM386

Защищенный режим в MS DOS использовался при запуске драйвера **emm386.sys** (**emm386.exe**) или **qemm.sys**. Фактически защищенный режим использовался не для защиты программ, по крайней мере в MS DOS, а для возможности доступа к дополнительной памяти.

EMM386 переводил процессор в защищенный режим, запуская ядро MS DOS в виртуальном режиме и транслируя ему вызовы от других задач MS DOS.

Полноценная реализация защищенного режима в MS DOS, DPML

EMM386 — это не полноценный защищенный режим в MS DOS: MS DOS по-прежнему выполняла задачи, написанные для 8086, поэтому для защищенного режима был разработан интерфейс DPML. Изначально DPML (DOS Protected Mode Interface) был предназначен для Windows 3.0 и применяется при запуске DOS-программ. В этом случае сервером DPML будет соответствующая реализация DPML в Windows.

Еще одна реализация DPML — DOS4/GW, ставшая самым популярным 32-битным расширителем DOS. Расширитель поставлялся в составе компилятора Watcom C и часто использовался в компьютерных играх (одна из известных DOOM). Другие производители компиляторов также поставляли расширители DOS.

Плоская модель памяти

Способ бессегментной адресации, когда регистр непосредственно указывает на область памяти, называется плоской моделью памяти (flat model). Такой способ адресации используется, например, в Linux. Впрочем, такой механизм не всегда бывает удобным. Так, при разработке виртуализации VMware специалисты столкнулись с проблемами при реализации 64-битного процессора от AMD. Из-за отказа от сегментации памяти не удалось сразу реализовать механизм работы виртуальной машины. Именно сегментация памяти использовалась для различения кода монитора и гостя.

Оценка загрузки LINUX

Ядро Linux изначально разрабатывалось для процессора 80386, поэтому проблем с совместимостью, которые возникли у MS DOS и Windows, у него не было.

Ядро Linux использует плоскую модель памяти. Фактически Linux работает с виртуальной памятью. То есть приложения в общем случае не знают, с какой областью памяти они работают: находится ли она в RAM или на диске. Приложениям доступна также расширенная память, которая находится в единственном экземпляре в RAM, но каждым приложением воспринимается как отдельная.

Подкачка виртуальной памяти реализуется через специальный раздел диска **swap**.

Для просмотра информации о памяти можно использовать команду:

```
sudo ps aux --sort %mem
```

Показывает значения:

- **RSS** (Resident Set Size) — сколько физической памяти RAM использует процесс; не включает память свопа и память из разделяемых библиотек (shared library), но включает стек и heap — динамически распределяемую память приложения;
- **VSZ** (Virtual Set Size) — показывает, сколько виртуальной памяти использует процесс; включает физическую память и swap + shared-память.

Если процесс A имеет 500K кода и задействует 2500K shared-библиотек, имеет 200K стека и кучи, из которых 100K в памяти (остальное в свопе), и имеет 1000K, загруженных из shared-библиотек, и 400K собственного кода, то:

- RSS: $400K + 1000K + 100K = 1500K$
- VSZ: $500K + 2500K + 200K = 3200K$

Однако часть памяти расшарена, многие процессы используют ее — и если сложить RSS всех процессов, получится, что памяти больше, чем имеется в системе. Поэтому существует параметр **PSS** (proportional set size). Он показывает разделяемую память пропорционально количеству запущенных процессов. Так, если бы процессов, использующих одну и ту же разделяемую память, было два, то показатель был бы следующим:

- PSS: $400K + (1000K/2) + 100K = 400K + 500K + 100K = 1000K$

Для запуска понадобится установить программу smem.

В Ubuntu это:

```
sudo apt-get install smem
```

Вывод **smem** похож на вывод **ps aux**. Например, посмотреть процент памяти, используемый пользователем:

```
sudo smem -u -p
```

В режиме X11 Server позволяет строить графики:

```
sudo smem --pie name -c "pss"
```

Утилита free

Утилита **free** выводит краткую информацию о памяти компьютера. По умолчанию выводит информацию в Кб.

```
$ free
```

```
buranov@ubuntu16:~$ free
              total        used        free      shared  buff/cache   available
Память:      1016024      179304      288992       10928       547728       644876
Подкачка:     1949692         296     1949396
```

Более удобным может оказаться вывод информации в Мб — в этом поможет ключ **m**.

```
$ free -m
```

```
buranov@ubuntu16:~$ free -m
              total        used        free      shared  buff/cache   available
Память:         992         175         282          10         534         629
Подкачка:       1903           0       1903
```

Рассмотрим, о чем нам говорит команда **free**.

Первая строка: память.

Столбцы:

- **total** — показывает, сколько памяти доступно операционной системе. Это общий объем памяти — тот, который занимает ядро;
- **used** — сколько памяти на данный момент задействовано приложениями;
- **free** — сколько памяти на данный момент невостребовано, то есть излишне для системы под такой нагрузкой. **Это не свободная память!**
- **shared** — показывает информацию о разделенной памяти (между процессами). Если два процесса используют одну библиотеку, она не будет дважды загружаться в память. Второму будет сказано, где находятся уже загруженные библиотеки;
- **buff/cache** — **buff** необходим для записи данных на диск. **Cache** — для чтения данных с диска. При работе с файлами приложения работают не с диском, а с оперативной памятью, выделенной ОС для работы с данными. Чем больше памяти выделено под буферы и кеши, тем быстрее будет работать система;
- **available** — сколько памяти доступно для приложений. Это поле информирует о данном параметре, а не **free**!

Система может освободить часть памяти под буферы и кеши. Это замедлит систему, но обеспечит ее работоспособность.

Расчет доступной для использования информации делается следующим образом:

Доступная память = free + ~70% от buff/cache

Если система видит много неиспользуемой памяти, она забирает ее под **buffer** и **cache**. И если мы видим некоторый объем в столбце **free**, это означает, что система все, что могла, под свои нужды (под **buffer** и **cache**) забрала, и эту память использовать не может — она излишняя.

Вторая строка: подкачка.

- **Total** — сколько всего информации может храниться в разделе подкачки.
- **Used** — сколько использовано.
- **Free** — сколько свободно.

```
buranov@ubuntu16:~$ free -m
```

	total	used	free	shared	buff/cache	available
Память:	992	175	282	10	534	629
Подкачка:	1903	0	1903			

Вся эта информация берется из файла **/proc/meminfo**:

```
cat /proc/meminfo
```

```
buranov@ubuntu16:~$ cat /proc/meminfo
MemTotal:        1016024 kB
MemFree:         289080 kB
MemAvailable:    645016 kB
Buffers:         40004 kB
Cached:          427700 kB
SwapCached:      108 kB
Active:          437660 kB
Inactive:        180920 kB
Active(anon):    78264 kB
Inactive(anon):  85944 kB
Active(file):    359396 kB
Inactive(file):  94976 kB
Unevictable:     3652 kB
Mlocked:         3652 kB
SwapTotal:       1949692 kB
SwapFree:        1949396 kB
Dirty:           0 kB
Writeback:       0 kB
AnonPages:       154440 kB
Mapped:          51400 kB
Shmem:           10928 kB
Slab:            80084 kB
SReclaimable:    58400 kB
SUnreclaim:      21684 kB
KernelStack:     3024 kB
PageTables:      3908 kB
NFS_Unstable:    0 kB
Bounce:          0 kB
WritebackTmp:    0 kB
CommitLimit:     2457704 kB
```

Практическое задание

Ответьте на следующие вопросы в Google Docs, максимально подробно объясняя, почему вы так считаете. Приложите ссылку на документ к уроку.

1. Что такое сегмент и смещение?
2. Почему в процессоре 8086 мы не могли использовать линейный (физический) адрес памяти?
3. С какой целью был создан защищенный режим адресации?
4. Что такое селектор?
5. Для чего была создана страничная адресация?
6. Как определить, сколько памяти свободно в ОС Linux?
7. Для чего нужно то, что хранится в **cache**?

8. Что хранится в **buffers**?
9. Показывает ли колонка **total** в утилите **free** весь объем физической памяти?
10. Что такое **SWAP**?

Задачи со * предназначены для продвинутых учеников.

Дополнительные материалы

1. [Введение в микропроцессор 80386.](#)
2. [htop и многое другое на пальцах.](#)
3. [Учебный курс. Часть 31. Сегментная адресация.](#)
4. [Кольца, уровни защиты.](#)
5. [Сегментация памяти в защищенном режиме.](#)
6. [Защищенный режим процессоров Intel 80286/80386/80486.](#)
7. [Как Linux работает с памятью.](#)
8. [Как Linux работает с памятью — 2.](#)
9. [Управление памятью в Linux.](#)
10. [What is RSS and VSZ in Linux memory management.](#)
11. [8086 в схемах.](#)
12. [Архитектура микропроцессора i8086.](#)

Используемая литература

Для подготовки данного методического пособия были использованы следующие ресурсы:

1. [Битовый сдвиг.](#)
2. [Сегментная адресация памяти.](#)
3. [LIFO.](#)
4. [Реальный режим.](#)
5. [X86-64.](#)
6. [80286.](#)
7. [80386.](#)